LINUX KERNEL
DESIGN PATTERNS

A Complete Reference for Kernel Internals, Architecture Patterns, Interview Preparation & Real-World Kernel Code

Covers: Callbacks · Singleton · Observer · Object Model · Vtables
Factory · Strategy · Chain of Responsibility · Template Method · Facade
Iterator · Reference Counting · RCU · Lock Patterns · Notifier Chains

Systems Software Engineering | Kernel Internals Reference | Interview Edition

1. Introduction — Why Patterns in the Kernel?

The Linux kernel is the largest, most battle-tested C codebase in existence — over 30 million lines developed across decades by thousands of engineers. It has no C++ virtual dispatch, no templates, no garbage collector. Yet it implements every classical design pattern seen in object-oriented systems, using nothing but raw C, function pointers, macros, and disciplined conventions.

Understanding these patterns is not academic. During kernel code review, every senior engineer will ask: why is this a function pointer? Why does this struct carry operations? Why is this list_head embedded rather than pointed to? The patterns are the answer.

> 📌 **NOTE**
> This document covers every major kernel design pattern: the C object model, callback/vtable systems, singleton, observer, factory, strategy, chain of responsibility, template method, facade, iterator, reference counting, RCU, and concurrency patterns. For each, you will see the real kernel struct, real kernel code, and concrete interview guidance.

1.1 The Kernel's C Object Model

The kernel simulates OOP in C through three core techniques:

- **Embedding structs:** A 'base class' is embedded as the first member of a 'derived class', allowing safe pointer casting between them.
- **Function pointer tables (ops structs):** A struct of function pointers represents a vtable. Any object that carries one can be treated polymorphically.
- **container_of macro:** Given a pointer to an embedded member, recover the pointer to the outer containing struct. This is the kernel's dynamic_cast.

```
// Linux Kernel C Object Model — include/linux/kobject.h
/* The fundamental kernel OOP idiom */

/* 'Base class' — embedded in every concrete type */
struct kobject {
    const char         *name;
    struct list_head    entry;
    struct kobject     *parent;
    struct kobj_type   *ktype;   /* vtable pointer */
    struct kset        *kset;
    struct kernfs_node *sd;
    struct kref         kref;   /* reference count */
    unsigned int        state_initialized:1;
    /* ... */
};

/* 'Derived class' — embeds kobject as first member */
struct device {
    struct kobject       kobj;  /* MUST be first */
    struct device       *parent;
    struct device_driver *driver;
    void                *driver_data;
```

```
    /* device-specific fields ... */
};

/* container_of: recover 'this' from embedded member */
#define container_of(ptr, type, member) \
    ({ const typeof(((type *)0)->member) *__mptr = (ptr); \
       (type *)((char *)__mptr - offsetof(type, member)); })

/* Usage: given a kobject*, get the enclosing device* */
struct kobject *kobj = get_some_kobject();
struct device  *dev  = container_of(kobj, struct device, kobj);
```

> ⚠️ **WARNING**
> container_of is type-safe in that it uses typeof() for compile-time member type checking. Passing the wrong member name is a compile error, not a silent cast. Never cast struct pointers directly — always use container_of.

2. Callback Pattern — Function Pointers as Virtual Dispatch

The most ubiquitous pattern in the kernel. A function pointer stored in a struct defers the decision of which function to call to runtime, exactly as a C++ virtual method does — but with full visibility and zero hidden overhead.

2.1 Anatomy of a Kernel Ops Struct (vtable)

Every major subsystem defines an ops struct. Drivers populate it; the core subsystem calls through it. This is the kernel's vtable mechanism.

```
// Virtual Filesystem Ops Struct — include/linux/fs.h
/* fs/vfs — The virtual filesystem vtable */
struct file_operations {
    struct module *owner;
    loff_t  (*llseek)   (struct file *, loff_t, int);
    ssize_t (*read)     (struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write)    (struct file *, const char __user *, size_t, loff_t *);
    __poll_t (*poll)    (struct file *, struct poll_table_struct *);
    long    (*unlocked_ioctl)(struct file *, unsigned int, unsigned long);
    int     (*open)     (struct inode *, struct file *);
    int     (*release)  (struct inode *, struct file *);
    int     (*fsync)    (struct file *, loff_t, loff_t, int datasync);
    /* ... 30+ more operations */
};

/* A concrete driver populates its ops struct */
static const struct file_operations my_driver_fops = {
    .owner          = THIS_MODULE,
    .open           = my_driver_open,
    .read           = my_driver_read,
    .write          = my_driver_write,
    .unlocked_ioctl = my_driver_ioctl,
    .release        = my_driver_release,
};

/* VFS core calls through the vtable — never knows the driver type */
static ssize_t vfs_write(struct file *file, const char __user *buf,
                         size_t count, loff_t *pos)
{
    /* NULL check before virtual dispatch — critical kernel pattern */
    if (!file->f_op->write && !file->f_op->write_iter)
        return -EINVAL;
    return file->f_op->write(file, buf, count, pos);
}
```

2.2 When ops pointers are NULL — The Null Object Pattern

An ops struct member being NULL is semantically meaningful: it means 'not supported'. The caller checks for NULL before dispatching. This is the kernel's Null Object pattern.

```
// Null-check before dispatch — fs/read_write.c
```

```
/* Checking NULL before virtual dispatch — from fs/read_write.c */
ssize_t do_iter_read(struct file *file, struct iov_iter *iter, loff_t *ppos,
                     rwf_t flags)
{
    size_t tot_len;
    ssize_t ret = 0;

    if (!(file->f_mode & FMODE_READ))
        return -EBADF;
    if (!(file->f_mode & FMODE_CAN_READ))
        return -EINVAL;

    /* Prefer read_iter if present, fall back to read */
    if (file->f_op->read_iter)
        ret = do_iter_readv_writev(file, iter, ppos, READ, flags);
    else if (file->f_op->read)
        ret = do_loop_readv_writev(file, iter, ppos, READ, flags);
    else
        return -EINVAL;   /* Neither op present: EINVAL */
    return ret;
}
```

2.3 Interrupt Handler Callback Chain

IRQ handlers are registered as callbacks. The kernel maintains a per-IRQ list of action structs. On interrupt, each handler is called in turn — if one handles it, it returns IRQ_HANDLED and the chain stops.

```
// IRQ Handler Registration — include/linux/interrupt.h
/* The interrupt action — a linked callback node */
struct irqaction {
    irq_handler_t       handler;     /* the callback itself */
    void               *dev_id;      /* opaque context pointer */
    struct irqaction   *next;        /* next in chain */
    unsigned long       flags;
    const char         *name;
    /* ... */
};

/* Registering a callback */
int request_irq(unsigned int irq, irq_handler_t handler,
                unsigned long flags, const char *name, void *dev)
{
    struct irqaction *action = kzalloc(sizeof(*action), GFP_KERNEL);
    if (!action) return -ENOMEM;

    action->handler = handler;
    action->dev_id  = dev;
    action->flags   = flags;
    action->name    = name;
    return __setup_irq(irq, desc, action);
}

/* A real interrupt handler callback */
static irqreturn_t my_irq_handler(int irq, void *dev_id)
{
    struct my_device *dev = dev_id;  /* recover typed context */

    if (!my_device_interrupted(dev))
        return IRQ_NONE;   /* not our interrupt */
```

```
    my_device_clear_irq(dev);
    wake_up(&dev->wait_queue);
    return IRQ_HANDLED;
}
```

💡 **INTERVIEW TIP**

Interview: 'What does IRQ_NONE mean?' → Your handler was called but the interrupt was not yours (shared IRQ line). Returning IRQ_NONE tells the kernel to keep calling the next handler in the chain. Returning IRQ_HANDLED stops the chain. If ALL handlers return IRQ_NONE the kernel prints a spurious interrupt warning.

3. Singleton Pattern — One Instance, System-Wide

The kernel has numerous system-wide singletons. Unlike user-space, these are not implemented with lazy initialization and locks in a getInstance() function. Instead, they are statically allocated globals initialized during boot, or they use init_once() patterns for deferred initialization.

3.1 Static Global Singletons

```
// Boot-time singletons — kernel/pid_namespace.c
/* The task_struct of PID 1 — global singleton pointer */
/* init/main.c */
struct task_struct *init_task;

/* The init mount namespace — one per boot */
struct mnt_namespace init_mnt_ns;

/* The initial PID namespace — root of all namespaces */
struct pid_namespace init_pid_ns = {
    .kref     = { .refcount = REFCOUNT_INIT(2) },
    .pid_cachep = NULL,  /* set at boot */
    .level    = 0,
    .child_reaper = &init_task,
};
```

3.2 Per-CPU Variables — the 'Distributed Singleton'

A critical kernel pattern: one logical variable, but a separate copy per CPU. Eliminates cache line bouncing and the need for locks on fast paths.

```
// Per-CPU Variables — include/linux/percpu.h
/* Declare a per-CPU variable */
DEFINE_PER_CPU(unsigned long, process_counts);
DEFINE_PER_CPU(struct softnet_data, softnet_data);  /* net/core/dev.c */

/* Access — must disable preemption to pin to current CPU */
void update_process_count(void)
{
    /* get_cpu() disables preemption, returns current CPU id */
    int cpu = get_cpu();
    per_cpu(process_counts, cpu)++;
    put_cpu();  /* re-enable preemption */
}

/* Shorthand for current CPU — same as get_cpu()/put_cpu() pair */
void update_fast_path(void)
{
    /* this_cpu_inc is atomic on the local CPU */
    this_cpu_inc(process_counts);
}

/* Iterate over all CPUs */
unsigned long total_count(void)
{
```

```
    unsigned long total = 0;
    int cpu;
    for_each_possible_cpu(cpu)
        total += per_cpu(process_counts, cpu);
    return total;
}
```

> 💡 **INTERVIEW TIP**
>
> Interview: 'Why use per-CPU variables instead of a single atomic?' → Atomics cause cache line bouncing: every CPU must acquire exclusive ownership of the cache line on each increment. Per-CPU variables keep data local to each CPU's cache. Only pay the cost of summing across CPUs when you need the global total — which is rare.

3.3 once_init — Lazy Singleton Initialization

```
// Lazy singleton initialization pattern
/* DEFINE_MUTEX + do_once pattern for lazy singletons */
static struct kmem_cache *my_cache;

static int init_my_cache(void)
{
    /* Executed exactly once across all callers, even concurrent */
    my_cache = kmem_cache_create("my_objects",
                                 sizeof(struct my_object),
                                 0, SLAB_HWCACHE_ALIGN, NULL);
    return my_cache ? 0 : -ENOMEM;
}

struct my_object *alloc_my_object(void)
{
    static int initialized = 0;
    /* kernel equivalent of call_once */
    if (unlikely(!initialized)) {
        if (init_my_cache() < 0)
            return NULL;
        WRITE_ONCE(initialized, 1);
    }
    return kmem_cache_alloc(my_cache, GFP_KERNEL);
}
```

4. Observer / Notifier Pattern — Decoupled Event Broadcasting

When one subsystem needs to broadcast events to an unknown set of listeners, the kernel uses notifier chains. The subject maintains a linked list of observer callbacks. Observers register and deregister dynamically. The subject fires events without knowing who is listening.

4.1 Notifier Chain Architecture

```
// Notifier chain types — include/linux/notifier.h
/* The notifier block — one per registered observer */
struct notifier_block {
    notifier_fn_t  notifier_call;  /* the observer callback */
    struct notifier_block *next;   /* intrusive linked list */
    int priority;                  /* higher = called first */
};

typedef int (*notifier_fn_t)(struct notifier_block *nb,
                             unsigned long action, void *data);

/* The chain head — owned by the subject */
struct blocking_notifier_head {
    struct rw_semaphore rwsem;
    struct notifier_block __rcu *head;
};
BLOCKING_NOTIFIER_HEAD(my_chain);
```

4.2 CPU Hotplug Notifier — Real Production Usage

```
// CPU Hotplug Observer — Real kernel module pattern
/* Observer: a subsystem that needs to know about CPU events */
static int my_cpu_callback(struct notifier_block *nb,
                           unsigned long action, void *hcpu)
{
    unsigned int cpu = (unsigned long)hcpu;

    switch (action) {
    case CPU_ONLINE:
        pr_info("CPU %u came online, allocating resources\n", cpu);
        if (alloc_per_cpu_resources(cpu) < 0)
            return NOTIFY_BAD;  /* veto the action */
        break;

    case CPU_DEAD:
        pr_info("CPU %u went offline, freeing resources\n", cpu);
        free_per_cpu_resources(cpu);
        break;

    case CPU_DOWN_PREPARE:
        /* Pre-down hook: migrate work off this CPU */
        drain_workqueue_cpu(cpu);
        break;

    default:
```

```
        return NOTIFY_DONE;  /* unhandled — pass to next observer */
    }
    return NOTIFY_OK;
}

static struct notifier_block my_cpu_nb = {
    .notifier_call = my_cpu_callback,
    .priority      = 10,   /* run before lower-priority observers */
};

/* Module init: register as observer */
static int __init my_module_init(void)
{
    return register_cpu_notifier(&my_cpu_nb);
}

/* Module exit: MUST deregister */
static void __exit my_module_exit(void)
{
    unregister_cpu_notifier(&my_cpu_nb);
}
```

> ⚠️ **WARNING**
>
> CRITICAL: Always unregister notifier blocks in module exit. A freed module's notifier_call pointer remaining in the chain will cause a kernel panic (NULL deref / use-after-free) on the next event. This is a very common driver bug.

4.3 Netdevice Notifier — Network Events

```
// Netdevice Notifier Observer — net/core/dev.c pattern
/* Subject fires: net/core/dev.c notifies on NETDEV_UP, DOWN, etc. */

/* Observer callback */
static int my_netdev_event(struct notifier_block *nb,
                           unsigned long event, void *ptr)
{
    struct net_device *dev = netdev_notifier_info_to_dev(ptr);

    switch (event) {
    case NETDEV_UP:
        netdev_info(dev, "Interface came UP\n");
        /* e.g., add ARP table entries, start traffic shaping */
        setup_traffic_shaper(dev);
        break;
    case NETDEV_GOING_DOWN:
        teardown_traffic_shaper(dev);
        break;
    case NETDEV_CHANGENAME:
        /* interface was renamed — update internal state */
        update_dev_name_cache(dev);
        break;
    }
    return NOTIFY_DONE;
}

static struct notifier_block my_netdev_nb = {
```

```
    .notifier_call = my_netdev_event,
};
register_netdevice_notifier(&my_netdev_nb);
```

Chain Type	Purpose & Usage
CPU events	register_cpu_notifier() — CPU hotplug (online/offline)
Network	register_netdevice_notifier() — interface up/down/rename
Reboot	register_reboot_notifier() — system shutdown/restart
PM events	register_pm_notifier() — suspend/resume lifecycle
Inet addr	register_inetaddr_notifier() — IPv4 address changes
Filesystem	fsnotify — inode event distribution to inotify/fanotify

5. Strategy Pattern — Interchangeable Algorithms

The Strategy pattern lets an algorithm be selected at runtime without changing the client. In the kernel, the canonical example is the I/O scheduler: the block layer is the client; the scheduler (CFQ, deadline, BFQ, mq-deadline, kyber) is the interchangeable strategy.

5.1 I/O Scheduler as Strategy

```
// I/O Scheduler Strategy Interface — block/elevator.h
/* The elevator_type: a strategy descriptor */
struct elevator_type {
    struct kmem_cache *icq_cache;
    struct elevator_mq_ops ops;  /* the strategy's op table */
    size_t icq_size;
    int icq_align;
    struct elv_fs_entry *elevator_attrs;
    const char *elevator_name;
    const char *elevator_alias;
    const unsigned int elevator_features;
    struct module *elevator_owner;
    /* ... */
};

/* Strategy operations (vtable for the scheduler) */
struct elevator_mq_ops {
    int  (*init_sched)(struct request_queue *, struct elevator_type *);
    void (*exit_sched)(struct elevator_queue *);
    int  (*init_hctx)(struct blk_mq_hw_ctx *, unsigned int);
    void (*exit_hctx)(struct blk_mq_hw_ctx *, unsigned int);
    bool (*allow_merge)(struct request_queue *, struct request *,
                        struct bio *);
    bool (*bio_merge)(struct request_queue *, struct bio *, unsigned int);
    int  (*request_merge)(struct request_queue *, struct request **,
                          struct bio *);
    void (*requests_merged)(struct request_queue *, struct request *,
                            struct request *);
    struct request *(*dispatch_request)(struct blk_mq_hw_ctx *);
    bool (*has_work)(struct blk_mq_hw_ctx *);
    /* ... */
};
```

5.2 Registering and Switching Strategies

```
// Strategy Registration and Dispatch — block/bfq-iosched.c
/* BFQ registers itself as an available I/O scheduling strategy */
static struct elevator_type bfq_sched_type = {
    .ops = {
        .init_sched        = bfq_init_queue,
        .exit_sched        = bfq_exit_queue,
        .bio_merge         = bfq_bio_merge,
        .dispatch_request  = bfq_dispatch_request,
        .has_work          = bfq_has_work,
        .allow_merge       = bfq_allow_bio_merge,
    },
```

```
    .icq_size    = sizeof(struct bfq_io_cq),
    .icq_align   = __alignof__(struct bfq_io_cq),
    .elevator_name  = "bfq",
    .elevator_owner = THIS_MODULE,
};

/* Runtime strategy switch via sysfs */
/* echo bfq > /sys/block/sda/queue/scheduler */

/* Block layer dispatches using current strategy — no if/else */
static blk_status_t blk_mq_dispatch_plug_list(struct blk_mq_hw_ctx *hctx)
{
    struct elevator_queue *e = hctx->queue->elevator;
    if (e && e->type->ops.dispatch_request)
        return e->type->ops.dispatch_request(hctx) ? BLK_STS_OK
                                                   : BLK_STS_RESOURCE;
    return blk_mq_do_dispatch_sched(hctx);
}
```

5.3 Network Congestion Control as Strategy

```
// TCP Congestion Control Strategy — net/ipv4/tcp_cubic.c
/* TCP congestion control ops — another strategy table */
struct tcp_congestion_ops {
    u32  (*ssthresh)(struct sock *sk);       /* get slow start threshold */
    void (*cong_avoid)(struct sock *, u32, u32); /* main CA algorithm */
    void (*set_state)(struct sock *, u8);    /* called on state change */
    void (*cwnd_event)(struct sock *, enum tcp_ca_event ev);
    u32  (*undo_cwnd)(struct sock *);
    struct tcp_info_ext *(*get_info)(struct sock *, u32, int *,
                                     union tcp_cc_info *);
    char  name[TCP_CA_NAME_MAX];
    struct module *owner;
};

/* CUBIC registers its strategy */
static struct tcp_congestion_ops cubictcp = {
    .init         = bictcp_init,
    .ssthresh     = bictcp_recalc_ssthresh,
    .cong_avoid   = bictcp_cong_avoid,
    .set_state    = bictcp_state,
    .undo_cwnd    = tcp_reno_undo_cwnd,
    .cwnd_event   = bictcp_cwnd_event,
    .pkts_acked   = bictcp_acked,
    .owner        = THIS_MODULE,
    .name         = "cubic",
};
```

6. Factory Pattern — Object Creation Without Specifying Type

The factory pattern separates object creation from usage. The kernel uses this in multiple places, most importantly in the VFS inode/superblock creation path and in the net socket creation path.

6.1 Filesystem Super Operations — inode Factory

```
// Inode Factory Method — fs/ext4/super.c
/* Filesystem superblock ops — alloc_inode is the factory method */
struct super_operations {
    struct inode *(*alloc_inode)(struct super_block *sb);
    void   (*destroy_inode)(struct inode *);
    void   (*free_inode)(struct inode *);
    void   (*dirty_inode)(struct inode *, int flags);
    int    (*write_inode)(struct inode *, struct writeback_control *wbc);
    void   (*drop_inode)(struct inode *);
    void   (*evict_inode)(struct inode *);
    void   (*put_super)(struct super_block *);
    int    (*sync_fs)(struct super_block *, int wait);
    /* ... */
};

/* ext4 factory: allocates an ext4_inode_info, returns embedded inode */
static struct inode *ext4_alloc_inode(struct super_block *sb)
{
    struct ext4_inode_info *ei;
    /* slab allocator creates the full ext4_inode_info */
    ei = alloc_inode_sb(sb, ext4_inode_cachep, GFP_NOFS);
    if (!ei)
        return NULL;
    /* Initialize ext4-specific fields */
    ei->vfs_inode.i_version = 1;
    spin_lock_init(&ei->i_raw_lock);
    INIT_LIST_HEAD(&ei->i_orphan);
    init_rwsem(&ei->xattr_sem);
    /* ... many more ext4-specific inits ... */
    return &ei->vfs_inode;  /* return upcast to VFS inode */
}

/* VFS calls the factory without knowing it's an ext4 inode */
struct inode *new_inode(struct super_block *sb)
{
    struct inode *inode = sb->s_op->alloc_inode(sb);
    /* ... generic inode setup ... */
    return inode;
}
```

6.2 Socket Creation Factory

```
// Socket Factory — net/ipv4/af_inet.c
/* net_proto_family: the socket factory for each protocol family */
struct net_proto_family {
    int    family;
    int  (*create)(struct net *net, struct socket *sock,
```

```
                    int protocol, int kern);
    struct module *owner;
};

/* AF_INET factory — creates an inet_sock */
static int inet_create(struct net *net, struct socket *sock,
                       int protocol, int kern)
{
    struct sock *sk;
    struct inet_protosw *answer;

    /* Look up the concrete protocol (TCP, UDP, RAW ...) */
    answer = inet_lookup_protocol(protocol, sock->type);
    if (!answer) return -EPROTONOSUPPORT;

    /* Factory creates the right sk type via proto->obj_size */
    sk = sk_alloc(net, PF_INET, GFP_KERNEL, answer->prot, kern);
    if (!sk) return -ENOBUFS;

    sock_init_data(sock, sk);
    sk->sk_prot->init(sk);  /* protocol-specific init */
    return 0;
}

/* User space: socket(AF_INET, SOCK_STREAM, 0) */
/* Kernel: inet_family->create() is called — user never sees the type */
```

6.3 kmem_cache — Object Pool Factory

```
// Slab Cache as Object Factory — kernel/fork.c
/* Create a typed object pool (factory for a specific type) */
static struct kmem_cache *task_struct_cachep;

void __init fork_init(void)
{
    /* Create a factory that produces task_struct objects */
    task_struct_cachep = kmem_cache_create_usercopy(
        "task_struct",
        arch_task_struct_size, align,
        SLAB_PANIC | SLAB_ACCOUNT,
        useroffset, usersize, NULL
    );
}

/* fork() uses the factory */
static struct task_struct *dup_task_struct(struct task_struct *orig, int node)
{
    struct task_struct *tsk;
    tsk = alloc_task_struct_node(node);  /* kmem_cache_alloc internally */
    if (!tsk) return NULL;
    /* copy from parent ... */
    return tsk;
}

/* Free back to factory */
static void free_task(struct task_struct *tsk)
{
    /* ... cleanup ... */
    kmem_cache_free(task_struct_cachep, tsk);
```

```
}
```

7. Template Method Pattern — Invariant Skeleton, Variable Steps

The Template Method pattern defines the invariant skeleton of an algorithm in a base function, while delegating the variable steps to callbacks or overridable functions. In the kernel, this is realized through probe/init/exit hooks and the platform driver model.

7.1 Platform Driver Lifecycle — The Classic Template

```
// Platform Driver Template Method — drivers/base/platform.c
/* The template: platform_driver defines invariant lifecycle hooks */
struct platform_driver {
    int  (*probe)(struct platform_device *);   /* variable step: init hw */
    void (*remove)(struct platform_device *);  /* variable step: teardown */
    void (*shutdown)(struct platform_device *);
    int  (*suspend)(struct platform_device *, pm_message_t state);
    int  (*resume)(struct platform_device *);
    struct device_driver driver;               /* base 'class' */
    const struct platform_device_id *id_table;
    bool prevent_deferred_probe;
};

/* INVARIANT SKELETON: platform_drv_probe (defined in drivers/base/platform.c)
 * This always runs for every matched device — the template */
static int platform_drv_probe(struct device *_dev)
{
    struct platform_driver *drv = to_platform_driver(_dev->driver);
    struct platform_device *dev = to_platform_device(_dev);
    int ret;

    /* Step 1 (invariant): set up pinctrl */
    ret = dev_pm_domain_attach(_dev, true);
    if (ret) goto out;

    /* Step 2 (variable): call driver's probe — the overridable step */
    if (drv->probe) {
        ret = drv->probe(dev);
        if (ret)
            dev_pm_domain_detach(_dev, true);
    }
out:
    return ret;
}

/* A concrete driver fills in the variable steps */
static int my_sensor_probe(struct platform_device *pdev)
{
    struct my_sensor *sensor;
    struct resource  *res;

    sensor = devm_kzalloc(&pdev->dev, sizeof(*sensor), GFP_KERNEL);
    if (!sensor) return -ENOMEM;

    res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
    sensor->base = devm_ioremap_resource(&pdev->dev, res);
    if (IS_ERR(sensor->base)) return PTR_ERR(sensor->base);

    sensor->irq = platform_get_irq(pdev, 0);
    devm_request_irq(&pdev->dev, sensor->irq,
```

```
                    my_sensor_irq, 0, "my_sensor", sensor);

    platform_set_drvdata(pdev, sensor);
    return 0;
}

static struct platform_driver my_sensor_driver = {
    .probe  = my_sensor_probe,
    .remove = my_sensor_remove,
    .driver = { .name = "my_sensor", .of_match_table = my_sensor_of_match },
};
module_platform_driver(my_sensor_driver);
```

💡 **INTERVIEW TIP**

Interview: 'What is devm_kzalloc and why do you use it?' → devm_ (device-managed) resources are automatically freed when the device is removed, even if probe() fails partway through. They are associated with the device via a devres list. This avoids memory leaks in error paths of probe(). Always prefer devm_ variants for driver resources.

7.2 Power Management Template

```
// Power Management Template Hooks
/* dev_pm_ops: another template method set for PM lifecycle */
struct dev_pm_ops {
    int (*suspend)(struct device *dev);
    int (*resume)(struct device *dev);
    int (*freeze)(struct device *dev);
    int (*thaw)(struct device *dev);
    int (*poweroff)(struct device *dev);
    int (*restore)(struct device *dev);
    int (*runtime_suspend)(struct device *dev);
    int (*runtime_resume)(struct device *dev);
    int (*runtime_idle)(struct device *dev);
    /* ... */
};

/* Helper macro — generates wrappers that handle NULL gracefully */
#define SIMPLE_DEV_PM_OPS(name, suspend_fn, resume_fn) \
const struct dev_pm_ops name = {                       \
    SET_SYSTEM_SLEEP_PM_OPS(suspend_fn, resume_fn)     \
}

/* Usage */
static SIMPLE_DEV_PM_OPS(my_sensor_pm_ops,
                         my_sensor_suspend,
                         my_sensor_resume);
```

8. Chain of Responsibility — Passing Requests Down a Chain

A request is passed along a chain of handlers until one handles it or the chain is exhausted. The kernel implements this in interrupt handling, netfilter, LSM (Linux Security Modules), and the VFS permission check path.

8.1 Netfilter Hook Chain

```
// Netfilter Chain of Responsibility — net/netfilter/nf_conntrack_proto.c
/* Each netfilter hook is a node in the chain */
struct nf_hook_ops {
    nf_hookfn       *hook;       /* the handler function */
    struct net_device *dev;
    void            *priv;       /* opaque context */
    u_int8_t         pf;         /* protocol family (NFPROTO_IPV4...) */
    unsigned int     hooknum;    /* which chain (PREROUTING/FORWARD...) */
    int              priority;   /* lower = earlier in chain */
};

/* Hook return values — control the chain */
/* NF_ACCEPT  : pass to next hook */
/* NF_DROP    : discard packet, stop chain */
/* NF_STOLEN  : hook owns packet, stop chain */
/* NF_QUEUE   : queue packet to userspace, stop chain */
/* NF_REPEAT  : re-call this hook */

/* A real netfilter hook — connection tracking pre-routing */
static unsigned int ipv4_conntrack_in(void *priv,
                                      struct sk_buff *skb,
                                      const struct nf_hook_state *state)
{
    return nf_conntrack_in(skb, state);
}

static const struct nf_hook_ops ipv4_conntrack_ops[] = {
    {
        .hook     = ipv4_conntrack_in,
        .pf       = NFPROTO_IPV4,
        .hooknum  = NF_INET_PRE_ROUTING,
        .priority = NF_IP_PRI_CONNTRACK,
    },
    {
        .hook     = ipv4_confirm,
        .pf       = NFPROTO_IPV4,
        .hooknum  = NF_INET_POST_ROUTING,
        .priority = NF_IP_PRI_CONNTRACK_CONFIRM,
    },
};
```

8.2 LSM (Linux Security Module) Stacking

LSM stacks multiple security modules (SELinux, AppArmor, SMACK) into a chain. Each security hook calls every registered LSM. Any LSM returning non-zero denies the operation.

```c
// LSM Security Hook Chain — security/security.c
/* LSM hook list — each entry is one security module's callback */
/* The chain is traversed for every security decision */

/* Simplified view of call_int_hook — iterate the LSM chain */
#define call_int_hook(FUNC, IRC, ...) ({                    \
    int RC = IRC;                                            \
    do {                                                     \
        struct security_hook_list *P;                        \
        hlist_for_each_entry(P, &security_hook_heads.FUNC, list) { \
            RC = P->hook.FUNC(__VA_ARGS__);                  \
            if (RC != 0)                                     \
                break;  /* First denial stops the chain */ \
        }                                                    \
    } while (0);                                             \
    RC;                                                      \
})

/* Called on every open() system call */
int security_file_open(struct file *file)
{
    /* Traverses: SELinux -> AppArmor -> SMACK -> ... */
    /* First non-zero return ends the chain with EPERM */
    int ret = call_int_hook(file_open, 0, file);
    if (ret) return ret;
    return fsnotify_open(file);
}
```

✅ Key Insight

The LSM chain is a Security Chain of Responsibility where any module can veto an operation. The kernel composites multiple independent security policies without any one policy knowing about the others. This is textbook Chain of Responsibility combined with the Open/Closed Principle.

9. Iterator Pattern — list_head and the Intrusive List

The kernel's most visible data structure pattern is the intrusive doubly-linked list. Unlike standard linked lists, the list_head is embedded inside the object. This means: one object can be on multiple lists simultaneously, no separate list node allocation, and O(1) access to the containing object via container_of.

9.1 list_head — The Intrusive List

```
// Intrusive List Iterator — include/linux/list.h
/* The list node — embedded INTO the object, not wrapping it */
struct list_head {
    struct list_head *next, *prev;
};

/* An object that lives on multiple lists simultaneously */
struct task_struct {
    /* ... hundreds of fields ... */
    struct list_head tasks;          /* global task list */
    struct list_head sibling;        /* parent's children list */
    struct list_head children;       /* this task's children */
    struct list_head thread_group;   /* thread group list */
    struct list_head ptraced;        /* ptrace child list */
    /* ... */
};

/* Iterating the task list — Iterator pattern */
void show_all_tasks(void)
{
    struct task_struct *task;

    rcu_read_lock();
    /* list_for_each_entry expands to a for loop with container_of */
    for_each_process(task) {   /* wraps list_for_each_entry on &init_task.tasks */
        pr_info("pid=%d comm=%s state=%ld\n",
                task->pid, task->comm, task->__state);
    }
    rcu_read_unlock();
}

/* The macro expands to: */
#define list_for_each_entry(pos, head, member)                    \
    for (pos = list_first_entry(head, typeof(*pos), member);      \
         !list_entry_is_head(pos, head, member);                  \
         pos = list_next_entry(pos, member))
```

9.2 hlist — Hash Table Lists

```
// Hash List Iterator — kernel/pid.c
/* hlist: singly-linked with a special head (saves 1 pointer per bucket) */
/* Used in hash tables where memory per-bucket matters */
struct hlist_head { struct hlist_node *first; };
struct hlist_node { struct hlist_node *next, **pprev; };
```

```
/* Process lookup hash table — pid_hash[] */
/* Each bucket is an hlist_head; each pid is an hlist_node */

struct pid {
    refcount_t count;
    unsigned int level;
    spinlock_t lock;
    struct hlist_head tasks[PIDTYPE_MAX];  /* tasks attached to this pid */
    struct hlist_node pidchain;            /* in the pid hash table */
    /* ... */
};

/* Finding a process by PID */
struct pid *find_pid_ns(int nr, struct pid_namespace *ns)
{
    struct pid *pid;

    hlist_for_each_entry_rcu(pid,
        &pid_hash[pid_hashfn(nr, ns)], pidchain) {
        if (pid->numbers[ns->level].nr == nr &&
            pid->numbers[ns->level].ns == ns)
            return pid;
    }
    return NULL;
}
```

9.3 RB-Tree Iterator — Ordered Iteration

```
// RB-Tree Iterator — kernel/sched/fair.c
/* CFS scheduler uses rb_tree — O(log n) insertion, O(1) leftmost */
struct cfs_rq {
    struct rb_root_cached tasks_timeline;  /* rb-tree of runnable tasks */
    /* ... */
};

struct sched_entity {
    struct rb_node   run_node;   /* node in the CFS rb-tree */
    struct list_head group_node;
    unsigned int     on_rq;
    u64              vruntime;   /* virtual runtime: tree key */
    /* ... */
};

/* Pick next task: leftmost node = smallest vruntime */
static struct sched_entity *__pick_first_entity(struct cfs_rq *cfs_rq)
{
    struct rb_node *left = rb_first_cached(&cfs_rq->tasks_timeline);
    if (!left) return NULL;
    return rb_entry(left, struct sched_entity, run_node);
}

/* In-order iteration over all runnable tasks */
static void debug_cfs_rq(struct cfs_rq *cfs_rq)
{
    struct rb_node *node;
    for (node = rb_first(&cfs_rq->tasks_timeline.rb_root);
         node; node = rb_next(node)) {
        struct sched_entity *se =
            rb_entry(node, struct sched_entity, run_node);
```

```
        pr_info("vruntime=%llu\n", se->vruntime);
    }
}
```

10. Reference Counting Pattern — Lifetime Management

In the kernel, objects cannot be freed while anyone holds a pointer to them. Reference counting is the primary mechanism for managing object lifetime. The kernel has evolved three generations: atomic_t, kref, and refcount_t (use-after-free hardened).

10.1 kref — The Standard Kernel Reference

```
// kref reference counting pattern — include/linux/kref.h
/* kref wraps refcount_t with get/put semantics */
struct kref {
    refcount_t refcount;
};

/* An object managed by kref */
struct my_device {
    struct kref  kref;   /* MUST be initialized before first get */
    void        *hw_base;
    int          irq;
    /* ... */
};

/* Destructor — called when refcount reaches zero */
static void my_device_release(struct kref *kref)
{
    struct my_device *dev = container_of(kref, struct my_device, kref);
    iounmap(dev->hw_base);
    free_irq(dev->irq, dev);
    kfree(dev);
}

/* Allocation — initialize with refcount = 1 */
struct my_device *my_device_create(void)
{
    struct my_device *dev = kzalloc(sizeof(*dev), GFP_KERNEL);
    if (!dev) return NULL;
    kref_init(&dev->kref);  /* refcount = 1 */
    return dev;
}

/* Get a reference — must be called before storing the pointer */
struct my_device *my_device_get(struct my_device *dev)
{
    kref_get(&dev->kref);
    return dev;
}

/* Release a reference — calls destructor when count reaches 0 */
void my_device_put(struct my_device *dev)
{
    kref_put(&dev->kref, my_device_release);
}
```

⚠ **WARNING**

CRITICAL: Never call kref_get() on an object that might already have refcount = 0. The object may be in the destructor. Use kref_get_unless_zero() when the object could be concurrently dying. This is mandatory for lock-free lookup patterns.

10.2 RCU — Read-Copy-Update for Lockless Reads

RCU is one of the kernel's most sophisticated synchronization mechanisms. It allows multiple readers to proceed without any locking, while writers make changes by replacing entire structures atomically. Readers never block; they simply defer to a grace period.

```
// RCU Read-Copy-Update — include/linux/rcupdate.h
/* RCU-protected pointer in a shared structure */
struct my_config {
    struct rcu_head  rcu;  /* required for call_rcu() */
    int              value;
    char             name[64];
};

/* Global RCU-protected pointer */
static struct my_config __rcu *current_config;

/* READER — no locks, just RCU read lock (disables preemption) */
int read_config_value(void)
{
    struct my_config *cfg;
    int val;

    rcu_read_lock();
    cfg = rcu_dereference(current_config);
    if (cfg)
        val = cfg->value;
    else
        val = -1;
    rcu_read_unlock();  /* no blocking between lock/unlock! */
    return val;
}

/* WRITER — allocates new, replaces atomically, frees old after grace period */
int update_config(int new_value, const char *new_name)
{
    struct my_config *new_cfg, *old_cfg;

    new_cfg = kmalloc(sizeof(*new_cfg), GFP_KERNEL);
    if (!new_cfg) return -ENOMEM;

    new_cfg->value = new_value;
    strscpy(new_cfg->name, new_name, sizeof(new_cfg->name));

    /* Atomically publish the new config */
    old_cfg = rcu_dereference_protected(current_config,
                  lockdep_is_held(&config_mutex));
    rcu_assign_pointer(current_config, new_cfg);

    /* Free old config after all readers finish their grace period */
    if (old_cfg)
        call_rcu(&old_cfg->rcu, free_config_rcu);
    return 0;
}
```

```
static void free_config_rcu(struct rcu_head *head)
{
    struct my_config *cfg = container_of(head, struct my_config, rcu);
    kfree(cfg);
}
```

💡 **INTERVIEW TIP**

Interview: 'What is a grace period in RCU?' → A grace period is the time after rcu_assign_pointer() during which at least one context switch has occurred on every CPU. After a grace period, it is guaranteed that no CPU is still inside an rcu_read_lock() section that saw the old pointer. Only then is it safe to free the old structure. call_rcu() defers the free callback until after one grace period.

11. Facade Pattern — Simplified Interface to a Complex Subsystem

A facade provides a simple, unified interface to a complex subsystem. The kernel uses facades extensively to hide subsystem complexity from drivers and from user space. The system call interface itself is the ultimate facade.

11.1 devm_ API — Resource Management Facade

The devm_ (device-managed resource) API is a facade over the devres subsystem. It hides the complexity of resource tracking, failure path cleanup, and teardown ordering behind simple allocation functions.

```
// devm_ Facade vs Manual Resource Management
/* Without facade — manual resource management in probe() */
static int probe_manual(struct platform_device *pdev)
{
    struct my_dev *dev;
    int ret;

    dev = kzalloc(sizeof(*dev), GFP_KERNEL);
    if (!dev) return -ENOMEM;

    dev->base = ioremap(res->start, resource_size(res));
    if (!dev->base) { ret = -ENOMEM; goto err_free_dev; }

    ret = request_irq(irq, handler, 0, "mydev", dev);
    if (ret) goto err_iounmap;

    ret = register_chrdev(...);
    if (ret) goto err_free_irq;

    return 0;

err_free_irq:  free_irq(irq, dev);
err_iounmap:   iounmap(dev->base);
err_free_dev:  kfree(dev);
    return ret;
}

/* With devm_ facade — cleanup is automatic, no goto chains */
static int probe_devm(struct platform_device *pdev)
{
    struct my_dev *dev;

    /* All of these are automatically freed on device removal */
    dev = devm_kzalloc(&pdev->dev, sizeof(*dev), GFP_KERNEL);
    if (!dev) return -ENOMEM;

    dev->base = devm_ioremap_resource(&pdev->dev, res);
    if (IS_ERR(dev->base)) return PTR_ERR(dev->base);

    return devm_request_irq(&pdev->dev, irq, handler, 0, "mydev", dev);
    /* If this returns error, all previous devm_ resources are freed */
}
```

11.2 regmap — Register Access Facade

regmap is a facade that provides a uniform API for accessing device registers regardless of the underlying bus (I2C, SPI, MMIO). Drivers call regmap_read/write; regmap handles bus-specific locking, caching, and format conversion.

```
// regmap Register Access Facade — drivers/base/regmap/
/* regmap_config — configure the facade for your device */
static const struct regmap_config my_sensor_regmap_config = {
    .reg_bits      = 8,     /* register address width */
    .val_bits      = 8,     /* register value width */
    .max_register  = 0x7F,
    .writeable_reg = my_sensor_writeable,
    .readable_reg  = my_sensor_readable,
    .volatile_reg  = my_sensor_volatile,
    .cache_type    = REGCACHE_RBTREE,  /* cache register values */
};

/* Probe: create the facade */
static int my_sensor_probe(struct i2c_client *client)
{
    struct my_sensor *sensor;
    sensor->regmap = devm_regmap_init_i2c(client,
                                          &my_sensor_regmap_config);
    if (IS_ERR(sensor->regmap))
        return PTR_ERR(sensor->regmap);
    return 0;
}

/* Driver code uses the facade — no I2C details visible */
int my_sensor_read_temp(struct my_sensor *s, int *temp)
{
    unsigned int raw;
    int ret;
    ret = regmap_read(s->regmap, REG_TEMPERATURE, &raw);
    if (ret) return ret;
    *temp = raw * 250 - 40000;  /* milli-Celsius */
    return 0;
}
```

12. Command / Workqueue Pattern — Deferred Execution

The Command pattern encapsulates a piece of work as an object that can be queued, deferred, or cancelled. The kernel's workqueue subsystem is the primary implementation. A work_struct is the command object; the workqueue is the executor.

12.1 work_struct — The Command Object

```
// Workqueue Command Pattern — include/linux/workqueue.h
/* work_struct — a command object for deferred work */
struct work_struct {
    atomic_long_t data;          /* internal state + flags */
    struct list_head entry;      /* queue linkage */
    work_func_t func;            /* the command: what to execute */
#ifdef CONFIG_LOCKDEP
    struct lockdep_map lockdep_map;
#endif
};

/* delayed_work: command with a timer */
struct delayed_work {
    struct work_struct  work;
    struct timer_list   timer;
    struct workqueue_struct *wq;
    int                 cpu;
};

/* Embedding work_struct in your device state */
struct my_device {
    struct work_struct  irq_work;    /* bottom half work */
    struct delayed_work poll_work;   /* periodic polling */
    struct workqueue_struct *wq;
    /* ... */
};

/* The command function — runs in a kernel thread context */
static void my_irq_work(struct work_struct *work)
{
    /* container_of to recover the device context */
    struct my_device *dev = container_of(work, struct my_device, irq_work);

    /* This runs in process context — can sleep, use mutexes */
    mutex_lock(&dev->lock);
    process_incoming_data(dev);
    mutex_unlock(&dev->lock);
}

/* Probe: initialize commands */
static int my_probe(struct platform_device *pdev)
{
    struct my_device *dev = devm_kzalloc(&pdev->dev, sizeof(*dev), GFP_KERNEL);

    /* Initialize the command objects */
    INIT_WORK(&dev->irq_work, my_irq_work);
    INIT_DELAYED_WORK(&dev->poll_work, my_poll_work);

    /* Create a private workqueue for this device */
    dev->wq = alloc_ordered_workqueue("my_device_wq", WQ_FREEZABLE);
```

```
    if (!dev->wq) return -ENOMEM;

    return 0;
}

/* IRQ handler (top half) — just schedules the command, returns fast */
static irqreturn_t my_irq_handler(int irq, void *data)
{
    struct my_device *dev = data;
    /* Queue the command — do NOT sleep here */
    queue_work(dev->wq, &dev->irq_work);
    return IRQ_HANDLED;
}

/* Schedule the polling command with a delay */
queue_delayed_work(dev->wq, &dev->poll_work, msecs_to_jiffies(100));
```

API / Flag	Purpose
`schedule_work()`	Queue work on system-wide shared workqueue
`alloc_workqueue()`	Create a private workqueue with specific attributes
`WQ_UNBOUND`	Work can run on any CPU — no CPU affinity
`WQ_FREEZABLE`	Work is frozen during system suspend
`WQ_HIGHPRI`	Work runs at high priority — use sparingly
`alloc_ordered_workqueue()`	Serialized execution — one work item at a time
`cancel_work_sync()`	Cancel and wait for running work to complete
`flush_workqueue()`	Wait for all pending work to drain

13. Inheritance / Type Hierarchy via Struct Embedding

Without C++ virtual inheritance, the kernel builds type hierarchies by embedding base structs at the start of derived structs. This allows safe upcast (pointer to derived = pointer to base) and downcast (container_of). The kobject/kset/ktype system is the full kernel class hierarchy.

13.1 The kobject / kobject_type Hierarchy

```
// kobject Type Hierarchy — include/linux/device.h
/*
 * Class Hierarchy:
 *
 *   kobject (abstract base)
 *     ├── device (via kobj embedded at offset 0)
 *     │      ├── platform_device (via dev embedded at offset 0)
 *     │      ├── pci_dev
 *     │      ├── usb_device
 *     │      └── net_device
 *     └── module_kobject
 */

/* Abstract base */
struct kobject {
    const char          *name;
    struct kobject      *parent;
    struct kobj_type    *ktype;   /* 'vtable' pointer */
    struct kref          kref;
    /* ... */
};

/* 'Virtual function table' for kobject — defines behavior */
struct kobj_type {
    void (*release)(struct kobject *kobj);       /* destructor */
    const struct sysfs_ops *sysfs_ops;           /* sysfs vtable */
    const struct attribute_group **default_groups;
    /* ... */
};

/* First-level derived class */
struct device {
    struct kobject kobj;      /* base class — MUST BE FIRST */
    struct device *parent;
    const struct device_type *type;   /* own vtable */
    struct device_driver *driver;
    void *driver_data;        /* private driver state */
    const struct attribute_group **groups;
    void (*release)(struct device *dev);
    /* ... */
};

/* Second-level derived class */
struct platform_device {
    const char   *name;
    int           id;
    struct device dev;      /* base 'class' — MUST BE FIRST */
    u64           platform_dma_mask;
    struct resource *resource;
    unsigned int  num_resources;
    /* ... */
```

```
};

/* Upcasting — implicit, always safe if base is first member */
struct platform_device *pdev = get_platform_device();
struct device *dev = &pdev->dev;        /* upcast */
struct kobject *kobj = &pdev->dev.kobj; /* double upcast */

/* Downcasting — explicit, via container_of */
struct device *dev = get_device_from_somewhere();
struct platform_device *pdev = to_platform_device(dev);
/* to_platform_device expands to: */
/* container_of(dev, struct platform_device, dev) */
```

13.2 net_device — The Network Inheritance Chain

```
// net_device Inheritance — include/linux/netdevice.h
/* net_device owns its own type hierarchy for operation polymorphism */
struct net_device {
    /* ... hundreds of fields ... */
    const struct net_device_ops *netdev_ops;  /* tx/rx/ioctl vtable */
    const struct ethtool_ops    *ethtool_ops; /* ethtool vtable */
    const struct header_ops     *header_ops;  /* L2 header vtable */
    /* ... */
};

/* Driver fills in its own ops struct */
static const struct net_device_ops my_netdev_ops = {
    .ndo_open         = my_open,
    .ndo_stop         = my_stop,
    .ndo_start_xmit   = my_xmit,
    .ndo_get_stats64  = my_get_stats,
    .ndo_set_mac_address = eth_mac_addr,  /* inherit from ethernet */
    .ndo_validate_addr   = eth_validate_addr,
};

/* Alloc with space for private data — like placement new */
static int my_probe(struct pci_dev *pci, ...)
{
    /* sizeof_priv appended after net_device — driver's private state */
    struct net_device *netdev =
        alloc_etherdev(sizeof(struct my_priv));
    struct my_priv *priv = netdev_priv(netdev);  /* access private part */
    netdev->netdev_ops = &my_netdev_ops;
    return register_netdev(netdev);
}
```

14. Concurrency Patterns — Locks, Barriers & Atomic Operations

Concurrency correctness is perhaps the most demanding aspect of kernel development. The kernel has a rich set of synchronization primitives, each with specific use cases. Using the wrong one is not a style issue — it is a correctness issue.

14.1 Lock Selection Guide

Primitive	When to Use
`mutex`	Sleepable process context only. Recursive acquisition = deadlock.
`spinlock_t`	Any context (including IRQ if _irqsave used). Must NOT sleep while held.
`rwlock_t`	Read-mostly data, multiple readers OR one writer. Spinlock-based.
`semaphore`	Counting semaphore. Rarely used — prefer mutex or completion.
`rw_semaphore`	Sleepable rwlock. Allows concurrent readers, exclusive writer.
`seqlock_t`	Read-mostly, writer never starves. Readers retry on concurrent write.
`atomic_t`	Single integer: inc/dec/cmpxchg without locks.
`completion`	One-shot event signaling between threads. Better than sem for this.
`RCU`	Read-mostly, frequent reads, infrequent writes. Readers never block.

14.2 Spinlock + IRQ Save — The Atomic Context Pattern

```
// Spinlock + IRQ Save Pattern — Process/IRQ Shared Data
/* Data shared between process context and IRQ handler */
struct my_ring_buffer {
    spinlock_t lock;
    u8  buf[4096];
    int head, tail;
    int count;
};

/* Writer in process context — must disable IRQs to prevent deadlock
 * If IRQ fires between spin_lock and spin_unlock, and the IRQ handler
 * also tries to acquire the same lock, we deadlock. */
int ring_buffer_write(struct my_ring_buffer *rb, u8 byte)
{
    unsigned long flags;
    int ret = 0;

    /* Save IRQ state AND acquire lock atomically */
    spin_lock_irqsave(&rb->lock, flags);

    if (rb->count < sizeof(rb->buf)) {
        rb->buf[rb->head] = byte;
        rb->head = (rb->head + 1) % sizeof(rb->buf);
        rb->count++;
    } else {
        ret = -ENOSPC;
    }
```

```
    spin_unlock_irqrestore(&rb->lock, flags);
    return ret;
}

/* Reader in IRQ handler — use spin_lock (not irqsave, already in IRQ) */
static irqreturn_t my_irq(int irq, void *data)
{
    struct my_ring_buffer *rb = data;
    u8 byte;

    spin_lock(&rb->lock);  /* IRQs already disabled in IRQ context */
    if (rb->count > 0) {
        byte = rb->buf[rb->tail];
        rb->tail = (rb->tail + 1) % sizeof(rb->buf);
        rb->count--;
    }
    spin_unlock(&rb->lock);
    return IRQ_HANDLED;
}
```

> ⚠ **WARNING**
> If any reader or writer of shared data runs in IRQ context, ALL accesses (including those in process context) MUST use spin_lock_irqsave/irqrestore. Using spin_lock alone in process context while the IRQ handler uses spin_lock will deadlock if the IRQ fires on the same CPU while the process context holds the lock.

14.3 Completion — One-Shot Event Signaling

```
// Completion Variable Pattern — IRQ to Process Signaling
/* struct completion: a binary semaphore for one-shot events */
struct my_async_op {
    struct completion  done;
    int                result;
};

/* Initiator: starts async work and waits */
int do_async_operation(struct my_device *dev)
{
    struct my_async_op op;
    init_completion(&op.done);

    dev->pending_op = &op;
    trigger_hardware_dma(dev);

    /* Sleep until IRQ handler signals completion */
    if (!wait_for_completion_timeout(&op.done, msecs_to_jiffies(1000)))
        return -ETIMEDOUT;

    return op.result;
}

/* IRQ handler: signals the waiter */
static irqreturn_t dma_irq_handler(int irq, void *data)
{
```

```
    struct my_device *dev = data;
    struct my_async_op *op = dev->pending_op;

    op->result = read_dma_result(dev);
    complete(&op->done);  /* wake the waiter */
    return IRQ_HANDLED;
}
```

15. Interview Cheat Sheet — Critical Questions & Answers

15.1 Canonical Interview Questions by Pattern

Object Model / container_of

```
Q: What does container_of do and when would you use it?
A: It recovers a pointer to the outer struct given a pointer to an embedded
   member. Used whenever a callback receives a pointer to an ops struct or
   list_head and needs to access the enclosing object.

Q: Why is kobject embedded at the start of device, and device at the start
   of platform_device?
A: When the base struct is the first member, its address equals the derived
   struct's address. This means you can upcast by identity (no offset), and
   container_of handles the downcast. Placing it anywhere else requires
   an offset in both directions.
```

Callbacks and Ops Structs

```
Q: Why check if an ops function pointer is NULL before calling it?
A: NULL means 'not implemented / not supported'. In the kernel Null Object
   pattern, NULL is the sentinel. Calling a NULL function pointer is an
   immediate kernel oops (NULL deref). The caller must gracefully handle
   the unsupported case, typically returning -EOPNOTSUPP or -EINVAL.

Q: What is the difference between file_operations.read and .read_iter?
A: read_iter uses iov_iter for scatter-gather I/O and is preferred for new
   drivers. The VFS will use read_iter if present, falling back to read.
   Implementing read_iter gives zero-copy paths and splice support for free.
```

Reference Counting

```
Q: When should you use kref_get_unless_zero vs kref_get?
A: kref_get_unless_zero when looking up an object from a hash/list under a
   lock where the object might concurrently be at refcount = 0 (dying).
   kref_get is only safe when you hold a reference already (or have a
   guarantee the object is alive).

Q: What is the purpose of rcu_head in an RCU-managed struct?
A: rcu_head is passed to call_rcu() along with the free callback.
   The kernel's RCU machinery uses it to defer the callback until a
   grace period — all ongoing RCU read sections have completed — has
   elapsed. Freeing before the grace period would cause use-after-free.
```

Concurrency

```
Q: Can you call mutex_lock from an interrupt handler?
A: No. Interrupt handlers run in atomic context; they cannot sleep.
   mutex_lock sleeps when the mutex is contended. Use spin_lock instead,
   and use spin_lock_irqsave if process context shares the same lock.

Q: What is the difference between READ_ONCE and ACCESS_ONCE?
A: ACCESS_ONCE is the old form, now deprecated. READ_ONCE ensures the
   compiler generates exactly one load from memory, preventing the
   compiler from caching the value in a register across a potential
   store by another CPU/thread. Always use READ_ONCE/WRITE_ONCE for
   variables shared between contexts without full locking.

Q: What does smp_mb() do and when is it needed?
A: smp_mb() is a full memory barrier — it prevents both the CPU and
   the compiler from reordering loads and stores across it. Needed
   when you must ensure another CPU sees writes in a specific order,
   e.g., writing data THEN setting a 'ready' flag. Without the barrier,
   the CPU may reorder, and the reader may see the flag before the data.
```

Workqueues

```
Q: When would you use alloc_ordered_workqueue vs alloc_workqueue?
A: ordered_workqueue serializes all work items — no two run concurrently.
   Use it when work items access shared state that would otherwise need
   locking. Regular workqueue allows parallel execution across CPUs.

Q: What is the difference between work_struct and tasklet?
A: tasklet runs in softirq (atomic) context — cannot sleep, very fast.
   work_struct runs in process context — can sleep, call mutexes, allocate
   with GFP_KERNEL. Prefer work_struct for new code unless you need
   the absolute minimum latency of softirq.
```

15.2 Anti-Patterns — What NOT to Do

> ⚠ **WARNING**
>
> NEVER call schedule(), msleep(), mutex_lock(), or any sleeping function from: IRQ handlers, softirq handlers, tasklets, timer callbacks, or RCU read-side critical sections.

> ⚠ **WARNING**
>
> NEVER use kfree() in an RCU read section on a pointer obtained via rcu_dereference(). The pointer may already be scheduled for freeing by a concurrent writer. Only readers increment the lifetime; call_rcu() handles the free.

> ⚠ **WARNING**

NEVER hold a spinlock across a function that may allocate memory with GFP_KERNEL. GFP_KERNEL may trigger direct reclaim which schedules — spinning with locks held through a schedule() is a hard deadlock.

💡 **INTERVIEW TIP**

Design Rule: The 'thin IRQ handler' principle. IRQ handlers should do the minimum necessary: acknowledge the hardware, save a few bytes of state, queue a work_struct, return IRQ_HANDLED. All real processing goes in the work function. This keeps interrupt latency low and allows the work to use the full process-context API.

16. Quick Reference — Pattern Summary Table

Pattern	Kernel Mechanism	Key Files	Interview Signal
`Callback/Vtable`	file_ops, net_device_ops, elevator_mq_ops	include/linux/fs.h	How does the VFS achieve polymorphism?
`Singleton`	init_task, per-CPU vars, DEFINE_PER_CPU	include/linux/percpu.h	Why per-CPU over atomic?
`Observer`	notifier_block chains, fsnotify	include/linux/notifier.h	CPU hotplug, netdev events
`Strategy`	I/O schedulers, TCP congestion	block/elevator.h	How to swap algorithms at runtime?
`Factory`	super_ops.alloc_inode, net_proto_family	include/linux/fs.h	How does ext4 allocate inodes?
`Template Method`	platform_driver probe/remove, dev_pm_ops	drivers/base/platform.c	devm_ lifecycle
`Chain of Resp.`	netfilter hooks, LSM stack	net/netfilter/	How does iptables intercept packets?
`Iterator`	list_for_each_entry, rb_entry	include/linux/list.h	Why intrusive over external lists?
`Reference Count`	kref, kref_put, call_rcu	include/linux/kref.h	kref_get_unless_zero usage
`RCU`	rcu_read_lock, rcu_assign_pointer	include/linux/rcupdate.h	Grace period definition
`Facade`	devm_, regmap, clk API	drivers/base/devres.c	Why devm_ over manual cleanup?
`Command/Deferred`	work_struct, delayed_work	include/linux/workqueue.h	Thin IRQ handler rule
`Inheritance`	kobject/device/platform_device	include/linux/kobject.h	container_of semantics
`Concurrency`	spinlock, mutex, seqlock, completion	include/linux/spinlock.h	Lock selection criteria